Training Deep Networks

Oct 13 | Fall 2026

Sit somewhere you can see the board!

Housekeeping

No cheat sheet for the midterm

We will release a practice midterm this week.

Discussion on Monday (10/19) will go over solutions. No quiz on 10/19.

Midterm in class on 10/20!

We will release more info about projects next week.

Quiz Review

```
w = torch.tensor(1.0, requires_grad=True)
optimizer = torch.optim.SGD([w], lr=0.5)

optimizer.zero_grad()
loss = 0.5 * (w - 3)**2
loss.backward()
optimizer.step()

loss = 0.5 * (w + 1)**2
loss.backward()
optimizer.step()
```

Quiz Review

1. Initialization

```
w = torch.tensor(1.0, requires_grad=True)
optimizer = torch.optim.SGD([w], lr=0.5)
```

No update yet.

`w = 1`

`w.grad` = None (not computed yet)

Quiz Review

1. Initialization

```
w = torch.tensor(1.0, requires_grad=True)
optimizer = torch.optim.SGD([w], lr=0.5)
```

No update yet.

`w = 1`

`w.grad = None` (not computed yet)

2. First gradient step

```
optimizer.zero_grad()
loss = 0.5 * (w - 3)**2
loss.backward()
optimizer.step()
```

Loss: $L = \frac{1}{2}(w-3)^2$

Gradient: $\frac{dL}{dw} = w - 3$

At $w = 1$: gradient $= 1 - 3 = -2$

SGD update: $w \leftarrow w - 0.5(-2) = 1 + 1 = 2$

After block 2:

`w = 2`

`w.grad = −2` (not cleared by step)

Quiz Review

1. Initialization

```
w = torch.tensor(1.0, requires_grad=True)
optimizer = torch.optim.SGD([w], lr=0.5)
```

No update yet.

`w = 1`

`w.grad = None` (not computed yet)

2. First gradient step

```
optimizer.zero_grad()
loss = 0.5 * (w - 3)**2
loss.backward()
optimizer.step()
```

Loss: $L = \frac{1}{2}(w-3)^2$

Gradient: $\frac{dL}{dw} = w - 3$

At $w = 1$: gradient $= 1 - 3 = -2$

SGD update: $w \leftarrow w - 0.5(-2) = 1 + 1 = 2$

After block 2:

`w = 2`

`w.grad = -2` (not cleared by step)

3. Second gradient step

```
loss = 0.5 * (w + 1)**2
loss.backward()
optimizer.step()
```

Loss: $L = \frac{1}{2}(w+1)^2$

Gradient: $\frac{dL}{dw} = w + 1$

At $w = 2$: gradient $= 2 + 1 = 3$

No `optimizer.zero_grad()`, so gradients **accumulate!**

Previous $w.\text{grad} = -2$

New gradient $= 3$

Accumulated gradient $= -2 + 3 = 1$

SGD update: $w \leftarrow 2 - 0.5(1) = 1.5$

After block 3:

`w = 1.5`

`w.grad = 1`

Today’s Agenda

1. How to initialize weights
2. Normalization
3. Minibatch noise

Today's Agenda

1. How to initialize weights
2. Normalization
3. Minibatch noise

Predicting Apartment Rent

Illustrative records

	Apartment A	Apartment B
		
Floor area	100 m^2	100 m^2
Bedrooms	1	2
Bathrooms	1	2
Distance to transit	1 km	0.2 km
Observed monthly rent	\$1,000 $(y = 1)$	\$3,000 $(y = 3)$

Predicting Apartment Rent

Illustrative records

	Apartment A	Apartment B
		
Floor area	100 m²	100 m²
Bedrooms	1	2
Bathrooms	1	2
Distance to transit	1 km	0.2 km
Observed monthly rent	\$1,000 $(y = 1)$	\$3,000 $(y = 3)$

For A: $x = (1, 1, 1, 1)$ For B: $x = (1, 2, 2, 0.2)$

Input features
Area / 100
Bedrooms
Bathrooms
Distance to transit (km)

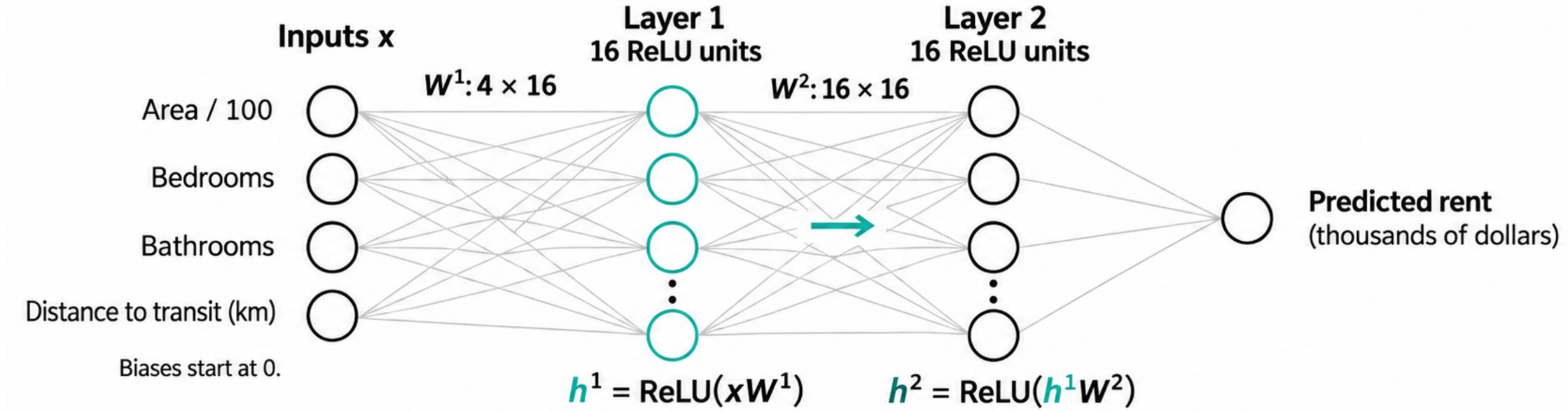
→

Rent model
$z = f_\theta(x)$

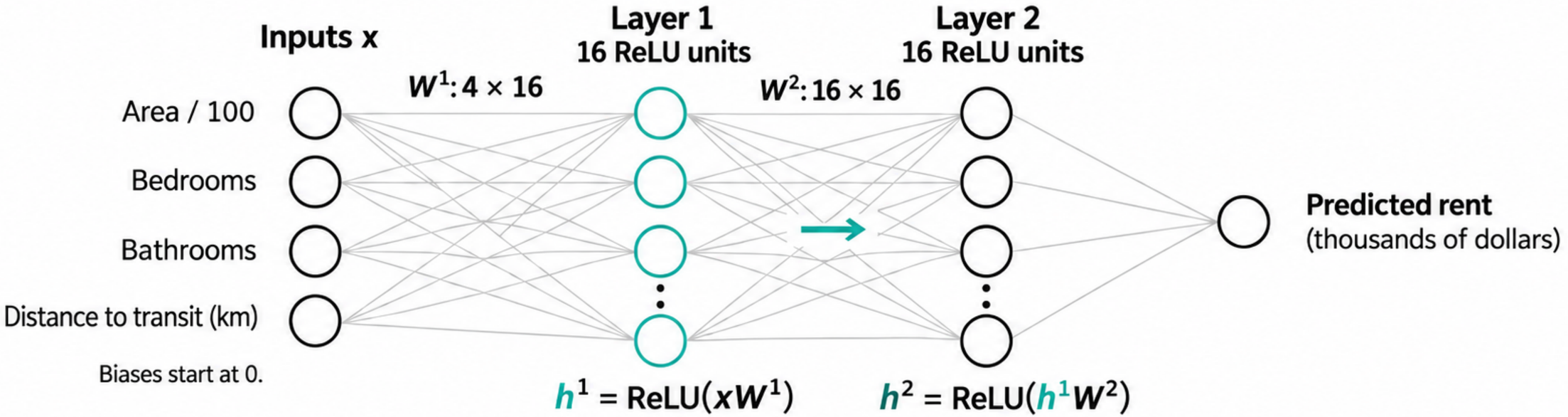
→

Predicted monthly rent
z
\$1,000 × z

How should we initialize the weights?

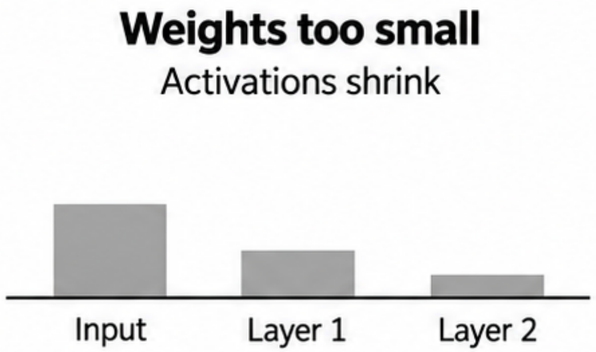


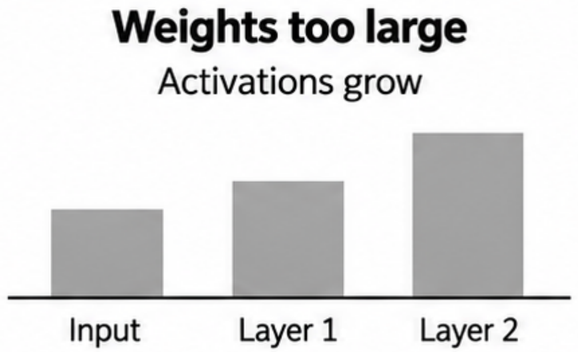
How should we initialize the weights?

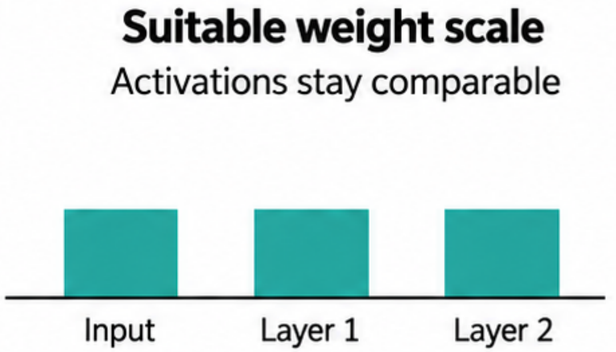


Activation size across layers

Starting weights in W^1 and W^2.

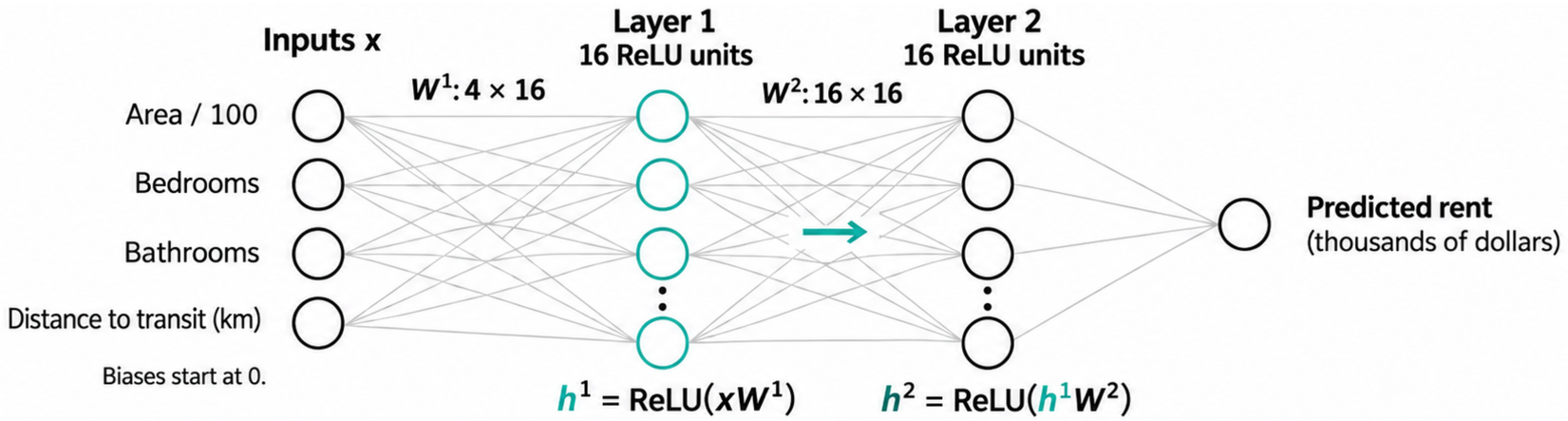






Measuring activation size

Assume weights are independent

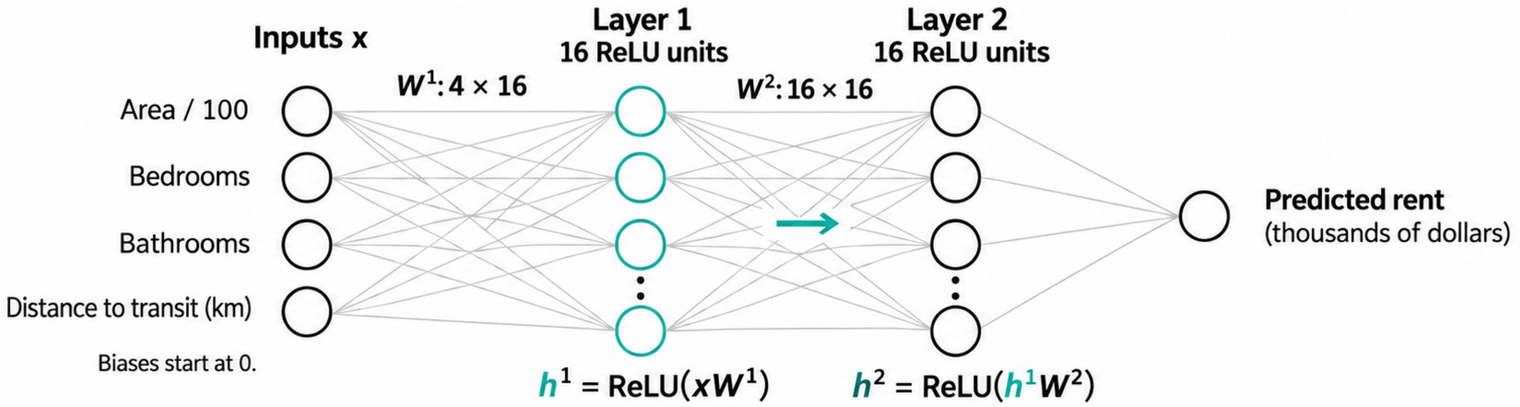


Activation size: squared norm per unit $q_i = \|h^i\|_2^2/m$ $m = 16$

Initial weights W^1, W^2 are random, so q_i is an expectation $q_i = \mathbb{E}[\|h^i\|_2^2]/m$

We want to pick initial weight distributions such that q_i is steady across different layers $q_{i-1} = q_i$

Initializing Weights



$$q_i = \frac{\mathbb{E}\left[\|h^i\|_2^2\right]}{m_i} = \frac{1}{m_i}\sum_{j=1}^{m_i}\mathbb{E}\left[(h_j^i)^2\right]$$

$$m_0 = 4, \qquad m_1 = m_2 = 16$$

$$q_0 = \frac{\|x\|_2^2}{4}$$

$$W_{kj}^1 \sim \mathcal{N}(0, s_1^2), \qquad b^1 = 0$$

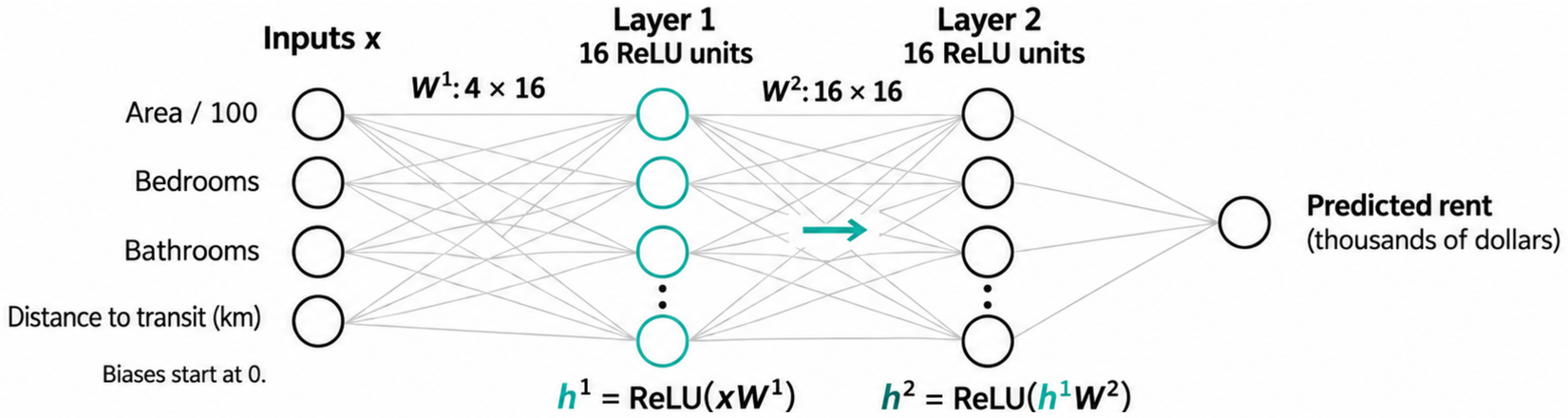
$$a^1 = xW^1$$

Calculation is on the chalkboard

Practice the next layer

$$W^2_{kj} \sim \mathcal{N}(0, s_2^2), \qquad b^2 = 0$$

He initialization (previous calculation): set weight variance to $2/d$ where d = number of incoming units



$$q_i = \mathbb{E}[\|h^i\|_2^2]/m$$

$$q_0 = 1$$

$$q_2 = \frac{16 s_2^2}{2} q_1$$

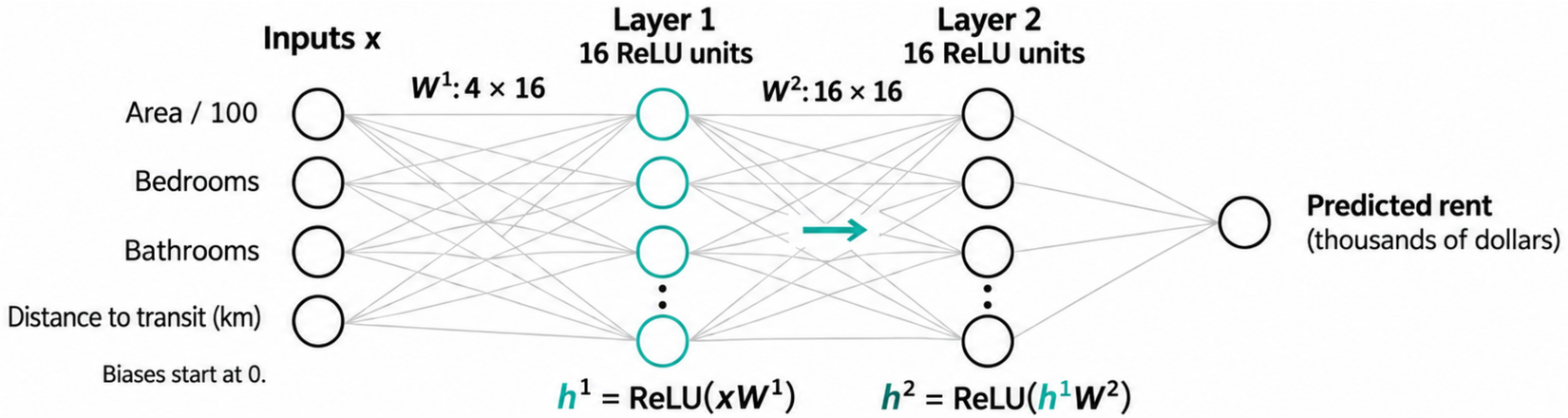
Starting weights	s_1^2	q_1 (given)	s_2^2	q_2
He initialization	$\frac{1}{2}$	1	_____	_____
Variance 1 in both layers	1	2	1	_____

Practice the next layer

$$W^2_{kj} \sim \mathcal{N}(0, s_2^2), \qquad b^2 = 0$$

He initialization (previous calculation): set weight variance to $2/d$ where $d =$ number of incoming units

$$q_2 = \frac{16 s_2^2}{2} q_1$$

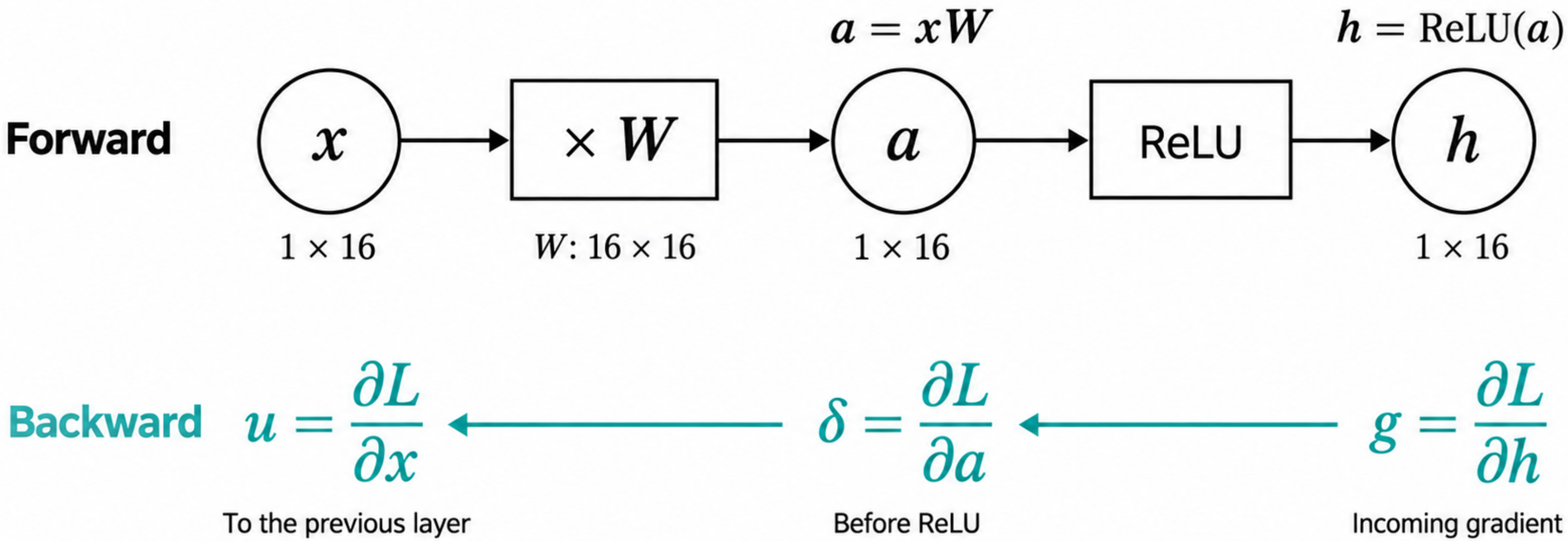


Starting weights	s_1^2	q_1 (given)	s_2^2	q_2
He initialization	$\frac{1}{2}$	1	$\frac{1}{8}$	1
Variance 1 in both layers	1	2	1	16

Does $q_2 = 1$ mean every activation is 1?

No, it's the expected squared norm per unit. It's probabilistic!

Backward pass!

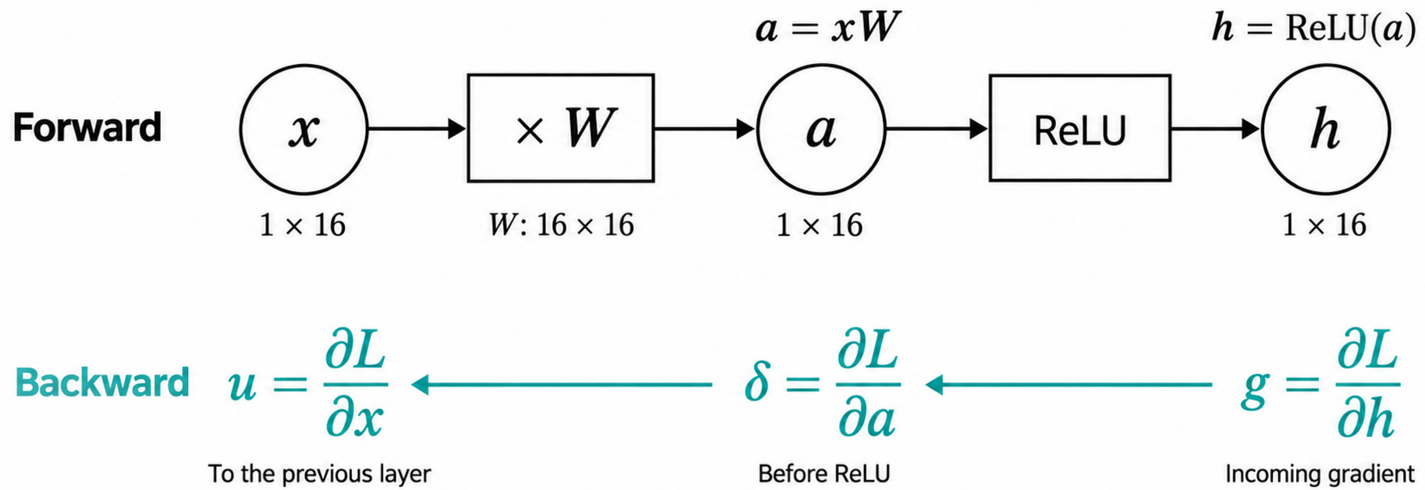


Chalkboard: what does He initialization do to the gradients?

$W^2_{kj} \sim \mathcal{N}(0, 2/16), \qquad b^2 = 0$

Also preserves the norms of the gradients!

Why does the gradient norm matter?



What happens if gradient norms shrink as we go backward through each layer? Grow?

Shrink: vanishing gradients; grow: exploding gradients

Does choosing a good initialization solve this problem?

No! Weights and activations change during training

Today’s Agenda

1. How to initialize weights
2. Normalization
3. Minibatch noise

Layer Normalization

Initialization ensures the scales are steady at the start of training

Normalization handles the activations during each forward pass of training

LayerNorm: normalize a vector across its coordinates

$$\text{LayerNorm}(h)_j = \gamma_j \frac{h_j - \mu}{\sqrt{v + \epsilon}} + \beta_j$$

γ, β are learned

μ is mean, v is variance

$h = (1,3)$ Compute the mean, variance, and normalized activation for $\gamma = 1, \beta = 0, \epsilon = 0$

$$\mu = \frac{1+3}{2} = 2$$

$$v = \frac{(1-2)^2 + (3-2)^2}{2} = 1$$

$$\text{LayerNorm}(h) = \left(\frac{1-2}{\sqrt{1}}, \frac{3-2}{\sqrt{1}} \right) = (-1,1)$$

What does a coordinate of -1 mean?

What information is preserved? What is lost?

Input standardization

Illustrative records

	Apartment A	Apartment B
		
Floor area	100 m^2	100 m^2
Bedrooms	1	2
Bathrooms	1	2
Distance to transit	1 km	0.2 km
Observed monthly rent	\$1,000 $(y = 1)$	\$3,000 $(y = 3)$

Each feature has different units and scales! Some can even be expressed in two ways (km vs miles)

Input standardization: normalize the inputs **across the dataset** so that they are all at the same scale

Input standardization

$$\hat{x} = \frac{x - \mu}{\sigma}$$

μ is mean, σ is standard deviation

Old network: $b + wx$

Find w', b' such that $b' + w'\hat{x} = b + wx$ for all x

$$w' = w\sigma \qquad b' = b + w\mu$$

Did standardization give our model a new representation of the data that it couldn't use before?

No! There exists a weight and bias such that we can make all the same predictions on normal data

So what's the benefit of standardization?

Gradient through input normalization

Compute first and second derivatives in each case. Suppose there is no bias.

$$\frac{1}{N}\sum_{i=1}^{N} x_i = 0, \qquad \sigma^2 = \frac{1}{N}\sum_{i=1}^{N} x_i^2, \qquad \hat{x}_i = \frac{x_i}{\sigma}$$

$$L(w) = \frac{1}{2N}\sum_{i=1}^{N} (wx_i - y_i)^2$$

$$\frac{\partial L}{\partial w} = \frac{1}{N}\sum_{i=1}^{N} (wx_i - y_i)x_i$$

$$\frac{\partial^2 L}{\partial w^2} = \frac{1}{N}\sum_{i=1}^{N} x_i^2 = \sigma^2$$

$$\widetilde{L}(v) = \frac{1}{2N}\sum_{i=1}^{N} (v\hat{x}_i - y_i)^2$$

$$\frac{\partial \widetilde{L}}{\partial v} = \frac{1}{N}\sum_{i=1}^{N} (v\hat{x}_i - y_i)\hat{x}_i$$

$$\frac{\partial^2 \widetilde{L}}{\partial v^2} = \frac{1}{N}\sum_{i=1}^{N} \hat{x}_i^2$$

$$= \frac{1}{\sigma^2}\left(\frac{1}{N}\sum_{i=1}^{N} x_i^2\right) = \frac{\sigma^2}{\sigma^2} = 1$$

Input normalization changes the curvature!

Allows us to use a large LR across heterogeneous features

Curvature across two features

Last lecture: $e_{j,t+1} = (1 - \eta H_{jj}) e_{j,t}, \qquad e_{j,t} = w_{j,t} - w_j^\star$

When the features are uncorrelated

Before standardization: $H_{11} = 100, \qquad H_{22} = 0.25, \qquad \eta = 0.01$

$$e_{1,t+1} = (1 - 0.01 \cdot 100) e_{1,t} = 0$$
$$e_{2,t+1} = (1 - 0.01 \cdot 0.25) e_{2,t} = 0.9975\, e_{2,t}$$

Reduce one coordinate to zero error
But other one has ~99% of its error still!

After standardization: $\widetilde{H}_{11} = \widetilde{H}_{22} = 1, \qquad \eta = 1$

$$\widetilde{e}_{j,t+1} = (1 - 1 \cdot 1) \widetilde{e}_{j,t} = 0, \qquad j = 1,2$$

Both coordinates can learn at the same rate

Today’s Agenda

1. How to initialize weights
2. Normalization
3. Minibatch noise

Minibatch gradients

When we draw a minibatch and compute loss + gradient, we are estimating the full dataset's gradient

$$g_b = L'(w) + \xi_b, \qquad \mathbb{E}[\xi_b] = 0, \qquad \mathrm{Var}(\xi_b) = \sigma^2$$

$$\bar{g} = \frac{1}{B}\sum_{b=1}^{B} g_b = L'(w) + \varepsilon, \qquad \varepsilon = \frac{1}{B}\sum_{b=1}^{B} \xi_b$$

Fresh noise for each example!

$$\mathbb{E}[\varepsilon] = ? \qquad \mathrm{Var}(\varepsilon) = ?$$

Minibatch gradients

When we draw a minibatch and compute loss + gradient, we are estimating the full dataset's gradient

$$g_b = L'(w) + \xi_b, \qquad \mathbb{E}[\xi_b] = 0, \qquad \mathrm{Var}(\xi_b) = \sigma^2$$

$$\bar{g} = \frac{1}{B}\sum_{b=1}^{B} g_b = L'(w) + \varepsilon, \qquad \varepsilon = \frac{1}{B}\sum_{b=1}^{B} \xi_b$$

Fresh noise for each example!

$$\mathbb{E}[\varepsilon] = \frac{1}{B}\sum_{b=1}^{B} \mathbb{E}[\xi_b] = 0$$

$$\mathrm{Var}(\varepsilon) = \frac{1}{B^2}\sum_{b=1}^{B} \mathrm{Var}(\xi_b)$$

$$= \frac{B\sigma^2}{B^2} = \frac{\sigma^2}{B}$$

You need to know these operations over variances, expectations, etc!

Increasing batch size reduces the amount of noise in the gradient

Tracking the error with noisy gradients

Recall our quadratic loss from before. How does minibatch noise change the update?

$$L(w) = \frac{\lambda}{2}(w - w^\star)^2, \qquad L'(w) = \lambda(w - w^\star)$$

$$g_t = \lambda(w_t - w^\star) + \varepsilon_t \qquad \mathbb{E}[\varepsilon_t] = 0, \qquad \mathrm{Var}(\varepsilon_t) = \frac{\sigma^2}{B}$$

$$w_{t+1} = w_t - \eta g_t$$

$$e_t = w_t - w^\star, \qquad M_t = \mathbb{E}[e_t^2]$$

Board work: $e_{t+1} = (1 - \eta\lambda)e_t - \eta\varepsilon_t$

$$e_{t+1}^2 = \ ?$$

$$M_{t+1} = \mathbb{E}[e_{t+1}^2] = \ ?$$

$$e_{t+1}^2 = (1 - \eta\lambda)^2 e_t^2 - 2\eta(1 - \eta\lambda)e_t\varepsilon_t + \eta^2\varepsilon_t^2 \,.$$

$$M_{t+1} = (1 - \eta\lambda)^2 M_t - 2\eta(1 - \eta\lambda)\mathbb{E}[e_t\varepsilon_t] + \eta^2\mathbb{E}[\varepsilon_t^2] \,.$$

Tracking the error with noisy gradients

$$M_{t+1} = \underbrace{(1-\eta\lambda)^2 M_t}_{\text{remaining error}} + \underbrace{\frac{\eta^2\sigma^2}{B}}_{\text{error added by noise}}$$

If we are at the minimum, will the next minibatch update keep us there?

Not necessarily - noise in the gradient can move us away

Why might an LR that works early in training be too large later?

As M_t gets smaller, second term dominates first term.
Reducing LR reduces the random movements
But reducing too much is going to slow us down too much!

LR decay: the practice of reducing LR over the course of training

Takeaways about optimization

How to compute gradient, Hessian, and curvature

How to set the learning rate

How to initialize the network so that learning is steady

Normalization as a way to keep activation scales steady

Minibatch noise throughout all of this

We have focused on quadratic objectives. This is not really deep learning!

You should use this intuition to understand optimization, but know
the key assumptions and when/why they might break!